

LAB 1: REQUIREMENTS ENGINEERING & UML USE-CASE MODELLING

Problem Statement #41 — OpenAPI Mock Server Generator

Name : T Surya

SRN : PES1UG24AM300

Problem Context: A developer productivity tool that ingests OpenAPI 3.0 YAML/JSON specifications, creates dynamic mock REST endpoints with schema validation, and simulates configurable response latency. Target stakeholders/actors: API Developer and QA Engineer.

1. Requirements Table

Functional Requirements — exactly 5 (FR-001 to FR-005)

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse OpenAPI 3.0 YAML/JSON schema definitions.	High	A valid OpenAPI 3.0 specification is parsed successfully; invalid syntax is rejected.	Parsing is necessary before endpoints and schemas can be used.
FR-002	Functional	The system shall generate dynamic HTTP mock endpoints from the parsed OpenAPI specification.	High	Each valid endpoint path and HTTP method can be accessed as a mock endpoint.	Dynamic endpoint generation is the core purpose of the tool.
FR-003	Functional	The system shall generate randomized JSON payloads that comply with the schemas defined in the OpenAPI specification.	High	Mock responses contain the correct schema-defined structure and data types.	Provides realistic schema-compliant API responses for development and testing.
FR-004	Functional	The system shall allow users to configure simulated response latency for mock endpoints.	Medium	The configured latency is applied to mock responses within the required tolerance.	Allows users to simulate different network/API response conditions.
FR-005	Functional	The system shall validate mock API requests and responses against the relevant	High	Valid requests/responses pass schema validation and	Supports API contract testing and QA validation.

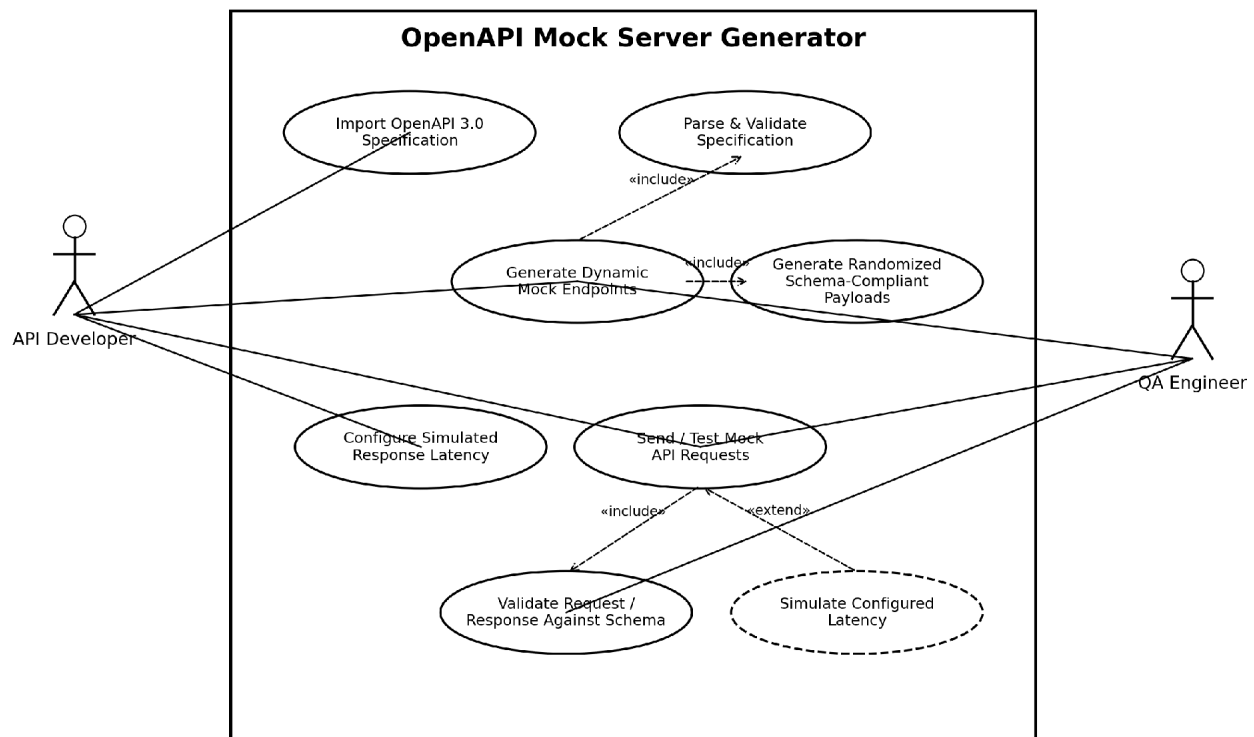
		OpenAPI schemas.		invalid ones are identified.	
--	--	------------------	--	------------------------------	--

Non-Functional Requirements — exactly 2 (NFR-001 & NFR-002)

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Performance	The mock server engine must sustain 1,000 mock API requests per second with user-configured simulated latency accurate within ± 10 ms.	High	Benchmarking tests confirm the target throughput and latency under simulated peak load.	Directly addresses the performance requirement specified in the problem statement.
NFR-002	Security	The mock server shall reject malformed or invalid API requests without exposing sensitive server information.	High	Security testing confirms invalid requests are rejected and implementation details are not leaked.	Supports safe operation of the mock server during development and QA testing.

2. UML Use-Case Diagram

The diagram models both required actors, the primary use cases, and includes both «include» and «extend» relationships.



3. Use-Case Flow Specification

Core Use Case: Generate Dynamic Mock Endpoints

Primary Actors: API Developer, QA Engineer

Preconditions

1. The OpenAPI Mock Server Generator is available and running.
2. The user has an OpenAPI 3.0 YAML/JSON specification.
3. The specification is accessible to the system.

Postconditions

4. The OpenAPI specification has been successfully parsed and validated.
5. Dynamic mock HTTP endpoints corresponding to the specification are generated.
6. Schema-compliant randomized JSON responses can be returned by the generated endpoints.

Main Success Scenario

7. The user imports an OpenAPI 3.0 YAML/JSON specification.
8. The system parses the specification.
9. The system validates the OpenAPI schema definitions and endpoint information.
10. The system identifies the API paths, HTTP methods, request/response schemas, and related definitions.

11. The system generates dynamic mock HTTP endpoints.
12. The system generates randomized JSON payloads that comply with the defined schemas.
13. The user sends a request to a generated mock endpoint.
14. The system validates the request/response against the relevant schema.
15. The system applies the configured simulated response latency, when configured.
16. The system returns the schema-compliant mock response to the user.

Alternate Flow

17. The user imports an OpenAPI 3.0 specification.
18. The system attempts to parse and validate the specification.
19. The system detects invalid syntax, an invalid endpoint definition, or an invalid schema.
20. The system rejects the invalid specification and reports the validation/parsing error.
21. The system does not generate the affected mock endpoints.
22. The user corrects the specification and imports it again.
23. The system reprocesses the corrected specification and continues with the Main Success Scenario.

Source basis: Problem Statement #41 supplied in the course PDF. The deliverables require exactly five functional requirements, two non-functional requirements, a UML use-case diagram with at least one «include» and one «extend» relationship, and a one-page use-case flow specification containing preconditions, postconditions, a main success scenario, and one alternate flow.